

UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

BRUNO LOURENÇO DOS SANTOS
GABRIEL CAVALCANTI ESMERALDO
JULIANA INACIO TEIXEIRA
MARCELO HENRIQUE MARQUES RIBEIRO
MATHEUS TEIXEIRA GOMES DE CARVALHO

DOCUMENTAÇÃO TÉCNICA

Aplicativo Prisma

FORTALEZA
2026

BRUNO LOURENÇO DOS SANTOS
GABRIEL CAVALCANTI ESMERALDO
JULIANA INACIO TEIXEIRA
MARCELO HENRIQUE MARQUES RIBEIRO
MATHEUS TEIXEIRA GOMES DE CARVALHO

DOCUMENTAÇÃO TÉCNICA

Aplicativo Prisma

Este documento contém a documentação técnica do Aplicativo Prisma desenvolvido na componente curricular N393 - Projeto Aplicado Multiplataforma como requisito para obtenção de nota.

Supervisor: Prof. Bruno Lopes, Me

FORTALEZA
2026

SUMÁRIO

1. INTRODUÇÃO....4
1.1. CONTEXTO E JUSTIFICATIVA.... 4
1.2. OBJETIVOS.... 4
1.3. ESCOPO E DELIMITAÇÃO.... 5
2. ENGENHARIA DE REQUISITOS....5
2.1. REQUISITOS FUNCIONAIS (RFs).... 5
2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)....6
3. PROJETO E ARQUITETURA DO SOFTWARE....8
3.1. ARQUITETURA GERAL.... 8
3.2 PROJETO DE DADOS: INTEGRAÇÃO COM API E PERSISTÊNCIA LOCAL.... 9
3.3. CONSUMO DA API E FLUXO DE NAVEGAÇÃO....10
4. TECNOLOGIAS E FERRAMENTAS.... 10
4.1. STACK DE TECNOLOGIAS....10
4.2. FERRAMENTAS DE DESENVOLVIMENTO....10

1. INTRODUÇÃO

1.1. CONTEXTO E JUSTIFICATIVA

O ecossistema de jogos eletrônicos é atualmente fragmentado entre grandes plataformas, como Steam, Xbox, PlayStation e RetroAchievements, cada uma possuindo seus próprios sistemas fechados de troféus, rankings e estatísticas. Para o jogador moderno, que frequentemente transita entre consoles e o PC, essa dispersão cria um cenário de dados difusos: não existe um local único onde ele possa visualizar seu desempenho global ou consolidar sua progressão de forma unificada. Essa fragmentação impede que o usuário tenha uma percepção clara de sua "carreira" no mundo dos games, resultando em conquistas que ficam esquecidas em perfis isolados.

O aplicativo Prisma surge como a solução para centralizar essa experiência, permitindo que jogadores unifiquem suas conquistas de múltiplas plataformas em um único perfil dinâmico. Além de oferecer uma visão holística do desempenho do usuário, o aplicativo permite a personalização do perfil para destacar conquistas favoritas e promove a interação social por meio de um sistema de "seguidores", conectando amigos e comunidades. O Prisma é essencial tanto para o jogador casual, que busca organizar suas memórias de jogo, quanto para o jogador profissional (e-athletes e streamers), funcionando como um portfólio mobile que proporciona visibilidade e comprova suas habilidades e histórico técnico em um mercado cada vez mais competitivo.

1.2. OBJETIVOS

O objetivo geral deste projeto é desenvolver um aplicativo mobile que integre e centralize o histórico de conquistas, troféus e estatísticas de diferentes ecossistemas de jogos (Steam, Xbox, PlayStation e RetroAchievements) para consolidar a identidade digital do jogador em um único perfil social e profissional.

Dito isso, os objetivos específicos do projeto são:

- Conectar o aplicativo mobile com o backend existente para consumo dos dados de conquistas e progresso em tempo real importados das principais plataformas de jogos;
- Implementar no aplicativo um perfil de usuário customizável que permite ao jogador selecionar e exibir suas conquistas de maior raridade ou importância (exibição de "badges" ou troféus favoritos);
- Implementar no aplicativo o sistema de seguidores, permitindo que usuários sigam outros perfis de jogadores;
- Implementar no aplicativo o sistema de ranking que pontua o desempenho dos jogadores somando os dados de todas as suas contas conectadas;

- Desenvolver uma interface mobile que mantenha a fidelidade visual e a identidade de design estabelecidas na versão web do Prisma, garantindo uma experiência contínua e fluida ao usuário.

1.3. ESCOPO E DELIMITAÇÃO

Nesta seção, definimos as fronteiras do aplicativo Prisma, detalhando as funcionalidades que serão entregues e estabelecendo claramente o que não faz parte dos objetivos deste desenvolvimento.

Escopo:

- **Integração e exposição de perfis:** Visualização centralizada de estatísticas das plataforma conectadas (Steam, Xbox, PlayStation, RetroAchievements);
- **Gestão de conquistas:** Sincronização e exibição do histórico de troféus obtidos pelo usuário em diferentes ecossistemas;
- **Personalização de perfil:** Funcionalidade para o usuário destacar suas conquistas favoritas;
- **Módulo de interação social:** Funcionalidade para seguir outros jogadores, ser seguido e navegar entre perfis da comunidade;
- **Navegação e busca:** Sistema de busca de usuários do Prisma.

Delimitação (Fora do Escopo):

- **Execução de jogos:** O aplicativo não é um emulador ou plataforma de execução; não será possível jogar através do Prisma;
- **E-commerce e transações:** Não haverá comercialização de jogos, itens in-game ou qualquer sistema de compra e venda (B2C/C2C);
- **Transmissão de áudio e vídeo:** O aplicativo não funcionará como plataforma de streaming ou canal de transmissão ao vivo (como Twitch ou YouTube).

2. ENGENHARIA DE REQUISITOS

2.1. REQUISITOS FUNCIONAIS (RFs)

Esta seção descreve os Requisitos Funcionais (RFs) do aplicativo Prisma.

ID	Nome do Requisito	Descrição

RF01	Criar conta	O aplicativo deve permitir que o usuário crie uma conta informando os seguintes dados: nome completo, email, nickname e senha.
RF02	Verificar email	O aplicativo deve validar o email do usuário durante o cadastro por meio de um código de verificação enviado ao endereço informado.
RF03	Realizar login	O aplicativo deve permitir que o usuário já cadastrado realize login ao informar email e senha.
RF04	Vincular contas externas	O aplicativo deve permitir que o usuário conecte suas contas da Steam, Xbox, PlayStation e RetroAchievements.
RF05	Sincronizar jogos e conquistas	O aplicativo deve requisitar a importação e atualização da lista de jogos e troféus obtidos nas plataformas conectadas.
RF06	Personalizar troféus fixados	O aplicativo deve permitir que o usuário selecione até 4 troféus específicos para destacar no topo de seu perfil.
RF07	Visualizar perfil	O aplicativo deve permitir que o usuário tenha uma visão geral do seu perfil, incluindo desempenho geral, conquistas e jogos recentes.
RF08	Seguir usuários	O aplicativo deve permitir que um jogador siga outros usuários e seja seguido.
RF09	Visualizar ranking geral	O aplicativo deve exibir a classificação geral de jogadores.
RF10	Personalizar perfil social	O aplicativo deve permitir a edição de foto de perfil, bio e nickname/username para exibição pública.

2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)

Esta seção descreve os Requisitos Não Funcionais (RNFs) do aplicativo Prisma.

- **Usabilidade:**

    - RNF01: A interface mobile deve ser responsiva e adaptável, permitindo que o jogador visualize seu portfólio de forma clara em diferentes tamanhos de tela e orientações de dispositivos smartphones.
    - RNF02: Antes de desvincular qualquer conta de plataforma (Steam, Xbox, PSN, RetroAchievements), o aplicativo exibe um modal de confirmação com a mensagem "Tem certeza que deseja desvincular sua conta?" e um aviso de que a ação não pode ser desfeita. O modal possui botões de confirmar e cancelar, além de suporte a fechamento por toque fora do modal, prevenindo perda acidental do histórico de sincronização.
    - RNF03: Durante a sincronização de múltiplas plataformas, o aplicativo deve exibir em tempo real o andamento da operação, com possibilidade de deixar essa visualização em segundo plano por meio do botão “Minimizar”.
    - RNF04: O aplicativo deve permitir que o usuário acesse qualquer tela principal do sistema (Perfil, Ranking, Seguidores e Configurações) com no máximo 2 toques a partir da tela inicial, utilizando preferencialmente uma barra de navegação inferior (Bottom Navigation).
    - RNF05: Telas que exibem grandes volumes de dados (como a lista completa de troféus de um jogo ou o ranking global) devem carregar os dados em lotes. O próximo lote deve ser requisitado automaticamente quando o usuário rolar a tela e atingir 80% do fim da lista atual.
- **Segurança:**
    - RNF06: O aplicativo deve implementar um controle de acesso restrito, impedindo a visualização de qualquer tela interna ou funcionalidade sem autenticação prévia, com exceção exclusiva das telas de login, cadastro de novos usuários e recuperação de senha.
    - RNF07: Toda requisição a endpoints internos da API (dashboard, perfil, configurações, sincronização) exige a presença de um token de autenticação válido associado ao usuário.
    - RNF08: A senha do usuário deve ser submetida ao algoritmo no momento da validação do changeset, antes de ser persistida no banco de dados. O campo password deve ser definido como virtual (virtual: true, redact: true) e removido do changeset após o hash, garantindo que nunca seja logado ou retornado em queries. O campo hashed_password deve armazenar apenas o hash, tornando a senha original irrecuperável mesmo em caso de vazamento do banco.
- **Compatibilidade:**
    - RNF09: O aplicativo deve ser totalmente funcional em dispositivos com sistema operacional Android a partir da versão 9.0 (API nível 28),

garantindo desempenho fluido e fidelidade aos componentes de design visual da versão web.

- **Portabilidade:**
    - RNF10: Todas as configurações sensíveis e específicas de ambiente (credenciais de banco, chave secreta, host, chaves de API de plataformas de jogos) devem ser definidas exclusivamente via variáveis de ambiente no arquivo config/runtime.exs, com validação de presença em produção, impedindo a inicialização caso alguma variável obrigatória esteja ausente. Isso permite que a mesma imagem Docker seja promovida entre ambientes sem recompilação.

3. PROJETO E ARQUITETURA DO SOFTWARE

3.1. ARQUITETURA GERAL

O aplicativo Prisma foi desenvolvido seguindo os princípios de Clean Architecture (Arquitetura Limpa) adaptada para o ecossistema React Native, adotando um fluxo de dados unidirecional (Unidirectional Data Flow). Esta arquitetura promove a separação estrita de responsabilidades, facilitando os testes, a escalabilidade e o isolamento entre a interface visual e as requisições de rede.

As camadas do aplicativo estão divididas e organizadas da seguinte forma:

1. **Camada de Apresentação (UI Layer):** Composta pelas pastas *app* e *components*. Utiliza o Expo Router para o roteamento baseado em arquivos (file-based routing) e o NativeWind/Tailwind CSS para a estilização. Esta camada é responsável exclusivamente por exibir os dados na tela e capturar as interações do usuário, delegando toda a lógica de estado e negócio para a camada inferior;
2. **Camada de Estado e Lógica (State/Logic Layer):** Composta pelas pastas *store* e *hooks*. Utiliza o Zustand para o gerenciamento do estado global de forma centralizada e performática. Os hooks customizados servem como ponte, conectando a interface visual aos dados e regras de negócio sem poluir os componentes visuais;
3. **Camada de Dados (Data Layer):** Concentrada na pasta *services*. Atua como a única fonte da verdade para a comunicação externa. Esta camada isola e centraliza todas as requisições e integrações com a API REST do backend do Prisma, entregando os dados formatados para a camada de estado;
4. **Camada de Domínio e Validação (Domain Layer):** Estruturada nas pastas *schemas*, *utils* e *constants*. Utiliza a biblioteca Zod para a definição de esquemas e validação estrita dos dados que entram e saem do aplicativo, integrando-se perfeitamente à tipagem estática do TypeScript. A pasta *utils* retém funções puras e regras de negócio que operam de forma independente do framework React.

A fazer: Diagrama da arquitetura

3.2. PROJETO DE DADOS: INTEGRAÇÃO COM API E PERSISTÊNCIA LOCAL

A estratégia de dados do aplicativo Prisma é projetada para garantir uma comunicação eficiente com o backend, utilizando a API REST como a fonte primária da verdade.Para o gerenciamento e cache local dos dados durante a execução do aplicativo, o projeto utiliza a biblioteca Zustand, que atua na camada de estado (*store*).

O fluxo de dados no aplicativo funciona da seguinte maneira:

1. **Requisição de dados:** A camada de *interface*, através dos hooks, solicita os dados necessários (como o perfil do usuário ou a lista de troféus);
2. **Consulta à API:** A camada de *services* centraliza e executa a requisição à API REST do backend do Prisma para buscar as informações mais recentes das plataformas de jogos conectadas.
3. **Armazenamento no estado global:** Ao receber a resposta da API, os dados são imediatamente salvos no estado global gerenciado pelo Zustand, que atua como um cache em memória leve e performático para o aplicativo;
4. **Exibição a partir do estado local:** Os componentes visuais da interface observam o estado armazenado no Zustand. Assim que os dados são atualizados no store global, a interface é renderizada automaticamente de forma reativa.

Essa abordagem garante alta performance na navegação, pois a interface é populada a partir do estado global em memória (Zustand), o que é extremamente rápido e evita requisições redundantes à internet ao transitar entre as telas do aplicativo. Toda a validação dos dados trafegados nesse fluxo é garantida de forma estrita pela biblioteca Zod.

Dicionário de Dados (Tabela `[pendente]`):
A fazer

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição

3.3. CONSUMO DA API E FLUXO DE NAVEGAÇÃO

A fazer

4. TECNOLOGIAS E FERRAMENTAS

4.1. STACK DE TECNOLOGIAS

Nesta seção listamos as stacks de tecnologia do aplicativo Prisma, detalhando os principais motivos para as escolhas feitas.

- **Linguagem:** TypeScript, por oferecer tipagem estática forte, prevenindo erros em tempo de compilação e garantindo maior manutenibilidade ao código;
- **Framework e Plataforma:** React Native e Expo, para o desenvolvimento mobile multiplataforma (Android e iOS) a partir de uma única base de código;
- **Roteamento e Navegação:** Expo Router, para o gerenciamento de navegação e fluxo entre telas baseado em estrutura de arquivos (file-based routing);
- **Estilização e UI:** NativeWind e Tailwind CSS, para a criação de interfaces declarativas, modernas e responsivas utilizando classes utilitárias;
- **Gerenciamento de Estado:** Zustand, utilizado no controle e compartilhamento global do estado da aplicação de forma leve e performática;
- **Validação de Dados:** Zod, para a definição de esquemas (schemas) e validação estrita de dados integrados ao TypeScript;
- **Animações:** Moti e React Native Reanimated, para a criação de microinterações e animações fluidas executadas na camada nativa;
- **Carregamento de Imagens:** Expo Image, para otimização, cache e renderização eficiente de mídias visuais no aplicativo.

4.2. FERRAMENTAS DE DESENVOLVIMENTO

Abaixo listamos as ferramentas que apoiaram o processo de desenvolvimento do aplicativo, passando pela escrita do código até o gerenciamento das tarefas da equipe.

- **IDE:** Visual Studio Code foi a IDE padrão para toda a equipa, devido à sua leveza, eficiência e extensibilidade para o ecossistema TypeScript e React Native;

- **Controle de Versões:** Git, com o repositório hospedado no GitHub. Adotamos um fluxo de trabalho baseado em branches independentes para cada membro desenvolver as suas tarefas antes de realizar o *merge* na `main`;

- **Gestão de Projeto:** Foi utilizado o quadro Kanban na área "Projetos" do GitHub, com as colunas "Backlog", "A Fazer", "Fazendo", "Em Revisão" e "Finalizado" para organizar as sprints e o desenvolvimento das funcionalidades;

- **Execução e Testes Mobile:** Expo Go, em conjunto com emuladores nativos (Android Studio / Xcode), para executar, testar e validar a aplicação diretamente em ambiente móvel em tempo real;

- **Gestão de Pacotes e Ambiente:** Node.js e o gestor de pacotes `npm` (ou `yarn`), utilizados para gerir a instalação de dependências e executar os scripts de inicialização e compilação do projeto.